

## تابع‌های لامبدا (Lambda) در پایتون

لامبدا یک روش ساده برای تعریف تابع در پایتون است. این توابع غالباً به نام «عملگرهای لامبدا» یا «تابع‌های لامبدا» نامیده می‌شوند.

پیش از آن‌که تابع‌های لامبدا را بررسی کنیم، می‌بایست به یک تابع بسیار مقدماتی که به روش سنتی تعریف شده است، نگاهی داشته باشیم:

```
def add_five(number):  
    return number + 5  
print(add_five(number=4))
```

```
$python3 main.py  
9
```

این تابع کاملاً ابتدایی است؛ اما به منظور نمایش کارکرد لامبداها ارائه شده است. تابعی که شما استفاده می‌کنید، ممکن است بسیار پیچیده‌تر از این باشد. این تابع به هر عددی که از طریق پارامتر `number` به آن ارسال می‌شود، 5 واحد اضافه می‌کند. تابع لامبدای معادل آن چنین است:

```
add_five = lambda number: number + 5  
print(add_five(number=4))
```

## تابع Map در پایتون چیست؟

تابع map در پایتون به عنوان یک ابزار سریع و کارآمد امکان اعمال یک تابع را بر روی تمام اعضای یک مجموعه داده بدون نوشتن حلقه‌های طولانی فراهم می‌کند.

map یکی از توابع داخلی قدرتمند و پرکاربرد زبان پایتون است که به شما امکان می‌دهد تا یک عملیات خاص (در قالب یک تابع تعریف می‌شود) را بر روی تک تک اعضای یک مجموعه داده (مانند لیست یا دیکشنری) اعمال کنید و نتایج جدید را در قالب یک مجموعه داده جدید دریافت کنید. به زبان ساده، تابع map همانند یک کارخانه است که یک دستورالعمل (تابع) را بر روی هر محصول (عنصر) از خط تولید (مجموعه داده) اعمال می‌کند و محصولات جدیدی را تحویل می‌دهد.

```
1 map(function, iterables)
```

با توجه به سینتکس ارائه شده در بالا تابع map دو آرگومان به عنوان ورودی دریافت می‌کند.

- **function**: تابعی را مشخص می‌کند که بر روی هر کدام از عناصر مجموعه داده اجرا می‌شود.
- **iterables**: این آرگومان یک شی قابل پیمایش مانند مجموعه داده، لیست، تاپل و سایر موارد دیگر را مشخص می‌کند.

## خروجی تابع map در پایتون

تابع map به عنوان خروجی یک شی از نوع map برمی‌گرداند که یک «شی قابل پیمایش (Iterator)» است. Iterator در پایتون یک شی قابل پیمایش است که به شما امکان می‌دهد تا عناصر یک مجموعه داده را یکی یکی و به ترتیب دریافت کنید. نکته مهم در خصوص خروجی تابع map در پایتون رفتار lazy آن است که یعنی تا زمانی که این شی را پیمایش نکنید یا به لیست یا تاپل تبدیل نکنید، هیچ عملی بر روی المان‌ها اجرا نمی‌شود. برای درک بهتر خروجی تابع map مثال زیر آورده شده است.

```
1 numbers = [1, 2, 3, 4]
2 mapped = map(lambda x: x * 2, numbers)
3 print(mapped)
```

در مثال بالا آرایه numbers به همراه تابع `lambda x: x * 2` به عنوان آرگومان ورودی به تابع map ارسال شده‌اند. با توجه به تابع داده شده انتظار می‌رود که هر کدام از عناصر آرایه numbers به توان ۲ برسند و مجدد آرایه جدیدی را ایجاد کنند. در صورت تبدیل نکردن خروجی تابع map به لیست، نتیجه زیر در خروجی نمایش داده خواهد شد.

```
map object at 0x7f8b2c4d5b50
```

برای مشاهده خروجی باید همانند قطعه کد زیر خروجی تابع `map` به یک لیست یا تاپل تبدیل شود.

```
1 numbers = [1, 2, 3, 4]
2 mapped = map(lambda x: x * 2, numbers)
3
4 print(list(mapped))
```

در نهایت خروجی تابع به صورت زیر خواهد بود.

```
[2, 4, 6, 8]
```

```
1 nums = [1, 2, 3]
2 as_tuple = tuple(map(str, nums))
3 print(as_tuple)
```

در مثال بالا آرایه `nums` به همراه تابع داخلی `str` به تابع `map` ارسال شده‌اند و خروجی نیز به نوع داده تاپل تبدیل شده است. تابع `str` در پایتون نوع داده عددی را به نوع داده رشته‌ای تبدیل می‌کند. خروجی حاصل از اجرای قطعه کد بالا به صورت زیر خواهد بود.

```
('1', '2', '3')
```

## تابع `map` در پایتون با تابع داخلی همراه با چندین آرگومان `iterable`

```
1 base_numbers = [2, 4, 6, 8, 10]
2 powers = [1, 2, 3, 4, 5]
```

با استفاده از تابع داخلی `pow()` و تابع `map` در پایتون قرار است که اعداد موجود در آرایه `base_numbers` به توان اعداد موجود در آرایه `powers` برسند. سپس، نتیجه حاصل در قالب آرایه جدیدی از اعداد نمایش داده شود.

```
1 numbers_powers = list(map(pow, base_numbers, powers))
2
3 print(numbers_powers)
```

در قطعه کد بالا تابع `map` بر روی هر کدام از عناصر موجود در آرایه‌های `base_numbers` و `powers` تابع داخلی `pow()` را اعمال می‌کند. بنابراین، خروجی حاصل از اجرای قطعه کد بالا به صورت زیر خواهد بود.

```
Output[2, 16, 216, 4096, 100000]
```

## کاربردهای تابع `map` در پایتون چیست؟

تابع `map` یک روش ساده و سریع برای اعمال یک تابع بر روی تک تک عناصر موجود در یک مجموعه داده است. مهم‌ترین کاربردهای تابع `map` عبارتند از:

- اعمال تغییرات ریاضی ساده
- تبدیل نوع داده‌ها
- پردازش چند لیست به صورت همزمان
- ترکیب با توابع از پیش تعریف شده
- پردازش فایل‌ها و داده‌های حجیم

```
1 def add_tax(price):  
2     return price * 1.09 # افزودن 9٪ مالیات  
3  
4 prices = [1000, 2000, 3000]  
5 taxed_prices = list(map(add_tax, prices))  
6 print(taxed_prices)
```

در مثال بالا تابع `add_tax()` به منظور اضافه کردن ۹ درصد مالیات بر قیمت تعریف شده است. این تابع به همراه آرایه `prices` به تابع `map` ارسال شده‌اند. خروجی حاصل از اجرای قطعه کد بالا به صورت زیر خواهد بود.

```
[1090.0, 2180.0, 3270.0]
```

```

1 numbers = [1, 2, 3]
2 result = []
3 for n in numbers:
4     result.append(n * 2) # ضرب هر عدد در ۲
5 print(result) # خروجی: [2, 4, 6]

```

در نقطه مقابل، تابع map برای اعمال یک تابع خاص بر روی تمام عناصر یک لیست مورد استفاده قرار می‌گیرد. همان مثال بالا با استفاده از تابع map به صورت زیر پیاده‌سازی خواهد شد.

```

1 numbers = [1, 2, 3]
2 result = list(map(lambda n: n * 2, numbers)) # اعمال تابع ضرب در ۲
3 print(result) # خروجی: [2, 4, 6]

```

هر کدام از ساختارهای map و حلقه for بسته به شرایط و نیازهای برنامه باید مورد استفاده قرار گیرند. در ادامه این دو ساختار از نظر ویژگی‌های زیر با یکدیگر مقایسه خواهند شد

| ویژگی        | تابع map                     | حلقه for                |
|--------------|------------------------------|-------------------------|
| کاربرد       | اعمال تابع بر روی تمام عناصر | تکرار روی عناصر مجموعه  |
| خوانایی      | بهتر برای عملیات ساده        | بهتر برای عملیات پیچیده |
| سرعت         | سریع‌تر                      | کندتر                   |
| خروجی        | شی map                       | ساختار دلخواه           |
| انعطاف‌پذیری | محدود                        | نامحدود                 |

## متد zip پایتون

فرض کنید ما دو لیست داریم ، یکی شامل نام ها و دیگری شامل نام خانوادگی است. ما می خواهیم به نوعی نامها را با نامهای خانوادگی مربوط به صورت چندتایی ترکیب کنیم. به عبارت دیگر ، ما می خواهیم عناصری از چندین نام تکرار شونده که دارای index یکسانی هستند را در لیستی از چندتایی ترکیب کنیم:

```
first_name = ['sam', 'brian', 'peter']
last_name = ['griffen', 'tucker', 'doe']
zipped_names = zip(first_name, last_name)
print(list(zipped_names))

[('sam', 'griffen'), ('brian', 'tucker'), ('peter', 'doe')]
```

ما می توانیم این کار را با تابع zip () که یک تابع پایتون داخلی است انجام دهیم. تابع zip () به دلیل مکانیزم مشابه زیپ های فیزیکی نامگذاری شده است. وقتی چیزی را فشرده می کنید، هر دو طرف را به هم نزدیک می کنید. و عملکرد zip () به این صورت است! این عناصر از index یکسانی از چندین شی iterable را به عنوان عناصر یکپارچه یکسان جمع می کند.

## استفاده از unzip به عنوان معکوس متد zip در پایتون

فرض کنید که لیست به مانند زیر دارید:

```
full_name = [('sam', 'griffen'), ('brian', 'tucker'), ('peter', 'doe')]
f_name, l_name = zip(*full_name)
f_name = list(f_name)
l_name = list(l_name)
print('first names are: ', f_name)
print('last names are:', l_name)

first names are: ['sam', 'brian', 'peter']
last names are: ['griffen', 'tucker', 'doe']
```

اگر بخواهید آیتم های این لیست را از هم جدا کنید می توانید متد zip را کنار علامت ستاره \* استفاده کنید:

## معرفی تابع Filter

با استفاده از تابع Filter می‌توانیم مقادیر مورد نظر خود را از یک توالی قابل پیمایش فیلتر کنیم. اولین آرگومان یک function object است که بر روی مقادیر آرگومان دوم که یک توالی است، اجرا خواهد شد و خروجی نهایی باید یک مقدار boolean باشد. اگر نتیجه‌ی اجرای Expression تعریف شده در function object بر روی هر کدام از مقادیر توالی قابل پیمایش برابر با True باشد، آن مقدار در لیست نهایی return می‌شود.

تفاوت تابع Filter با تابع Map در این است که شما می‌توانید چندین توالی قابل پیمایش را به تابع Map پاس دهید اما در تابع Filter فقط می‌توانید یک توالی را وارد کنید.



```
filter(function_object, iterable)
```

بیا ببینیم عددهای زوج را از میان لیست [1, 2, 3, 4, 5, 6] فیلتر کنیم:

```
l = [1, 2, 3, 4, 5, 6]
even_nums = filter(lambda x : x % 2 == 0, l)
print(list(even_nums))
```

```
[2, 4, 6]
```

## تابع Reverse در پایتون چیست ؟

مانند انواع دیگر «دنباله‌های قابل تغییر (Mutable Sequence)» در پایتون، در لیست‌های پایتون می‌توان از تابع Reverse استفاده کرد. این روش، از پایه همه داده‌های موجود در لیست را در حافظه برنامه معکوس می‌کند. در ادامه مثالی برای نمایش روش استفاده از متد Reverse در پایتون ارائه شده است:



```
1 >>> digits = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2
3 >>> digits.reverse()
4 >>> digits
5 [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

زمانی که تابع `Reverse` در پایتون برای یک لیست موجود فراخوانی می‌شود، داده‌های این لیست در همان مکان در حافظه معکوس خواهند شد. بنابراین پس از این عملیات، زمانی که برنامه نویس به لیست احتیاج داشته باشد، فقط معکوس این لیست موجود است. باید به این نکته توجه شود که تابع `Reverse` در پایتون یک لیست جدید را برنمی‌گرداند؛ برای مشخص شدن این موضوع لازم است به کدهای زیر توجه شود:

```
1 >>> digits = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2
3 >>> reversed_digits = digits.reverse()
4 >>> reversed_digits is None
5 True
```

- تابع `reverse()` در پایتون هیچ [آرگومانی](#) را در پرانتزهای مقابلش دریافت نمی‌کند.
  - تابع `Reverse` در پایتون هیچ مقداری را برنمی‌گرداند، در واقع این تابع مقدار ورودی را برزورسانی می‌کند.
  - اگر هر چیزی به جز لیست با تابع `Reverse` در پایتون مورد استفاده قرار بگیرد، این تابع دچار خطای «`AttributeError`» می‌شود.
- برای پیاده‌سازی عملیات معکوس کردن لیست با استفاده از قابلیت اندیس‌دهی منفی در پایتون، می‌توان از یک حلقه `For` و تابع `range` در یک دوم ابتدایی لیست با استفاده از `len(digits) // 2` (طول نصف لیست مربوطه) استفاده کرد. سپس، از یک عبارت انتساب به صورت موازی جهت جابه‌جایی عناصر استفاده می‌شود. در ادامه کدهای این مثال ارائه شده‌اند :

```
1 >>> digits = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2
3 >>> for i in range(len(digits) // 2):
4 ...     digits[i], digits[-1 - i] = digits[-1 - i], digits[i]
5 ...
6
7 >>> digits
8 [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

در این مثال یک لیست حاوی رشته ابتدا در خروجی چاپ می‌شود. سپس این لیست با استفاده از متد `reverse` در پایتون معکوس شده و نتیجه آن هم در خروجی چاپ شده است.

```
1 # Operating System List
2 systems = ['Windows', 'macOS', 'Linux']
3 print('Original List:', systems)
4
5 # List Reverse
6 systems.reverse()
7
8
9 # updated list
10 print('Updated List:', systems)
```

خروجی کدهای فوق پس از پیاده‌سازی به صورت زیر نمایش داده می‌شوند:

```
Original List: ['Windows', 'macOS', 'Linux']
Updated List: ['Linux', 'macOS', 'Windows']
```

در این مثال دو لیست به عنوان ورودی دریافت می‌شوند که لیست‌هایی از اعداد و حروف هستند و معکوس آن‌ها در خروجی چاپ می‌شود. کدهای این مثال در ادامه نمایش داده شده‌اند:

```
1 # Python3 program to demonstrate the
2 # use of reverse method
3
4 # a list of numbers
5 list1 = [1, 2, 3, 4, 1, 2, 6]
6 list1.reverse()
7 print(list1)
8
9 # a list of characters
10 list2 = ['a', 'b', 'c', 'd', 'a', 'a']
11 list2.reverse()
12 print(list2)
```

در ادامه خروجی کدهای فوق نمایش داده شده است:

```
[6, 2, 1, 4, 3, 2, 1]
['a', 'a', 'd', 'c', 'b', 'a']
```

## sort در پایتون چیست؟

تابع sort در پایتون برای مرتب‌سازی عناصر موجود در لیست‌ها به کار برده می‌شود. این تابع می‌تواند عناصر درون لیست را به صورت صعودی یا نزولی مرتب کند. متد sort() خروجی ندارد. یعنی اینکه وقتی بر روی لیستی اعمال می‌شود، به جای ساخت لیست جدیدی از عناصر مرتب شده، تمام داده‌های درون همان لیست را به شکل مرتب می‌چیند.

## مرتب‌سازی صعودی با `sort` در پایتون

در حالت پیش‌فرض، متد `sort()` تمام عناصر لیست را به شکل صعودی مرتب می‌کند. در کادر پایین، نمونه‌ای از مرتب‌سازی با کمک این متد را پیاده‌سازی کرده‌ایم.

```
1 numbers = [34, 1, 9, 5, 22]
2 numbers.sort()
3 print(numbers)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نمایش داده می‌شود.

```
[1, 5, 9, 22, 34]
```

## مرتب‌سازی نزولی با `sort` در پایتون

همین‌طور که در بالا گفتیم این متد به صورت پیش‌فرض تمام عناصر لیست را در حالت صعودی مرتب می‌کند. اما اگر بخواهیم که این عناصر به شکل نزولی مرتب شوند باید از آرگومان `reverse=True` در متد استفاده کنیم. در کادر پایین روش استفاده از این آرگومان را نمایش داده‌ایم.

```
1 numbers = [34, 1, 9, 5, 22]
2 numbers.sort(reverse=True)
3 print(numbers)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نمایش داده می‌شود.

```
[34, 22, 9, 5, 1]
```

## استفاده از پارامتر key در sort در پایتون

پارامتر key در متد sort() به برنامه نویسان کمک می کند که عملیات مرتب سازی را با توجه به معیارهای مدنظر خودشان تنظیم کنند. به پارامتر key باید تابع اختصاص بدهیم. وظیفه تابع این است که مقداری را برای مرتب سازی برگرداند. بیشتر اوقات در این قسمت از تابع lambda استفاده می کنیم.

```
1 # Sorting based on Length of the string
2 strings = ["banana", "apple", "cherry"]
3 strings.sort(key=len)
4 print(strings)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نمایش داده می شود.

```
['apple', 'banana', 'cherry']
```

## مرتب سازی سفارشی با استفاده از کلید

با کمک متد sort() می توانیم ساختارهای داده پیچیده تر را نیز مرتب کنیم. برای مثال، فرض کنید که باید لیستی از [دیکشنری ها](#) را مرتب کنیم. برای انجام این کار از کلیدهای دیکشنری ها کمک می گیریم. با کمک این کلیدها داده ها را هر طور که بخواهیم مرتب می کنیم. به منظور اجرای این عملیات باید از پارامتر key در متد sort() استفاده کنیم.

```
1 users = [{'name': 'Bob', 'age': 30}, {'name': 'Alice', 'age': 25}]
2 users.sort(key=lambda user: user['age'])
3 print(users)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نمایش داده می شود.

```
[{'name': 'Alice', 'age': 25}, {'name': 'Bob', 'age': 30}]
```

## مرتب‌سازی محصولات در فروشگاه‌های دیجیتال

فرض کنیم که در حال نوشتن برنامه‌ای برای پلتفرم‌های [تجارت الکترونیک](#) به زبان پایتون هستیم. در چنین نرم‌افزارهایی با کمک متد `sort()` می‌توانیم تمام اجناس را بر اساس قیمت، نام یا ویژگی‌های دیگر مرتب کنیم.

```
1 product_prices = [('T-shirt', 20), ('Jeans', 45), ('Socks', 5)]
2 product_prices.sort(key=lambda item: item[1]) # Sorts by price
3 print("Products sorted by price:", product_prices)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نمایش داده می‌شود.

```
Products sorted by price: [('Socks', 5), ('T-shirt', 20), ('Jeans', 45)]
```

## سیستم‌های نمره دهی در مدرسه یا دانشگاه

نرم‌افزارهای آکادمیک یا تحصیلی با کمک مرتب‌سازی نمره دانش‌آموزان آن‌ها را رتبه‌بندی می‌کنند. بیشتر اوقات، عملیات مربوط به مرتب‌سازی از شرایط مبتنی بر مقادیر [Boolean](#) به عنوان بخشی از منطق کار خود استفاده می‌کنند.

```
1 grades = [("Student A", 88), ("Student B", 92), ("Student C", 85)]
2 grades.sort(key=lambda x: x[1], reverse=True) # Sorts by score descending
3 print("Student rankings:", grades)
```

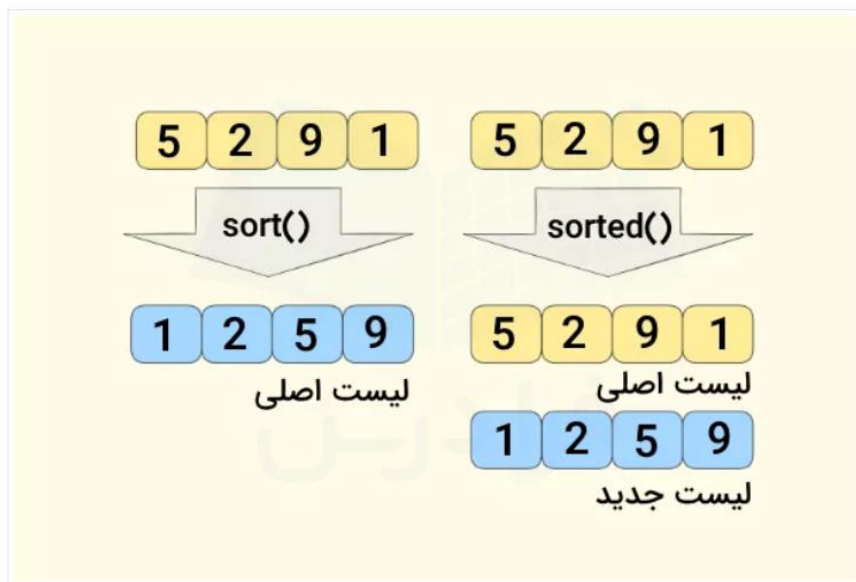
بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نمایش داده می‌شود.

```
Student rankings: [('Student B', 92), ('Student A', 88), ('Student C', 85)]
```

## مقایسه بین sorted و sort در پایتون

وظیفه اصلی تابع `sort()` این است که داده‌های درون لیست را تغییر بدهد. بنابراین خروجی ندارد، بلکه لیست اصلی را اصلاح می‌کند. اما پایتون تابع دیگری هم به نام `sorted()` دارد. این تابع عناصر هر شی پیمایش‌پذیری را که دریافت کند به صورت مرتب شده در لیست جدیدی ذخیره کرده و در خروجی برمی‌گرداند.

به این رفتار که لیست اصلی را بدون ساختن لیست جدید تغییر بدهیم در اصطلاح پایتون «In-Place» گفته می‌شود. یعنی می‌گوییم که تابع `sort()` از نوع توابع In-Place است. در عوض تابع `sorted()` به لیست اصلی دست نمی‌زند. یعنی ترتیب عناصر درون آن را تغییر نمی‌دهد. بلکه لیست جدیدی به صورت مرتب شده می‌سازد.



مقایسه تفاوت عملکرد بین متدهای `sort` و `sorted` در پایتون